3 DevOps Weekend Projects

Detailed How-To Guide — Commands, Code & Expected Outputs

Project 1: CI/CD with Azure DevOps | Project 2: Docker + Kubernetes on AKS | Project 3: Terraform + Azure

PROJECT 1: CI/CD End-to-End Deployment with Azure DevOps

Automatically build, test, and deploy a Node.js app to Azure App Service on every git push.

STEP 1 Create Your Node.js Application

Create a project folder and initialize a Node.js Express app:

```
mkdir cicd-demo && cd cicd-demo
npm init -y
npm install express
```

Create the file app.js with this content:

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => res.send('Hello from Azure DevOps Pipeline! v1'));
app.get('/health', (req, res) => res.json({ status: 'ok', version: '1.0' }));

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
module.exports = app;
```

Create a basic test file app.test.js:

```
const request = require('supertest');
const app = require('./app');

test('GET /health returns ok', async () => {
  const res = await request(app).get('/health');
  expect(res.statusCode).toBe(200);
  expect(res.body.status).toBe('ok');
});
```

Install test libraries and add test script to package.json:

```
npm install --save-dev jest supertest
# In package.json, set: "test": "jest"
```

✓ EXPECTED OUTPUT

```
node app.js → Server running on port 3000
curl localhost:3000/health → {"status":"ok","version":"1.0"}
npm test → PASS app.test.js
```

STEP 2 Dockerize the Application

Create Dockerfile in the project root:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Create .dockerignore to keep image small:

```
node_modules
.git
*.test.js
.env
```

Build and test the Docker image locally:

```
docker build -t cicd-demo:v1 .
docker run -d -p 3000:3000 --name cicd-test cicd-demo:v1
curl http://localhost:3000/health
docker stop cicd-test && docker rm cicd-test
```

✓ EXPECTED OUTPUT

```
Successfully built abc123def456
Successfully tagged cicd-demo:v1
curl → {"status":"ok","version":"1.0"}
```

STEP 3 Push Code to GitHub

Initialize Git and push to a new GitHub repository:

```
git init
git add .
git commit -m "Initial commit: Express app with Dockerfile"
# Go to github.com → New Repository → name: cicd-demo
git remote add origin https://github.com/YOUR_USERNAME/cicd-demo.git
git branch -M main
git push -u origin main
```

✓ EXPECTED OUTPUT

```
Branch 'main' set up to track remote branch 'main' from 'origin'
All files visible on github.com/YOUR_USERNAME/cicd-demo
```

STEP 4 Create Azure App Service

Use Azure CLI to create the App Service (or use Azure Portal):

```
# Login to Azure
az login

# Create a Resource Group
az group create --name cicd-rg --location eastus

# Create an App Service Plan (free tier)
az appservice plan create \
  --name cicd-plan \
  --resource-group cicd-rg \
  --sku F1 \
  --is-linux

# Create the Web App (Node.js 18)
az webapp create \
  --name cicd-demo-app-YOUR_INITIALS \
  --resource-group cicd-rg \
  --plan cicd-plan \
  --runtime 'NODE|18-lts'
```

⚠ NOTE: App name must be globally unique. Replace YOUR_INITIALS with something unique.

✓ EXPECTED OUTPUT

```
App created: https://cicd-demo-app-YOUR_INITIALS.azurewebsites.net
State: Running (shows default Azure page for now)
```

STEP 5 Create Azure DevOps Project & Connect to GitHub

Go to dev.azure.com → Sign in with your Microsoft/GitHub account.

- Click '+ New Project' → Name: cicd-demo → Visibility: Private → Create
- Left sidebar → Pipelines → Pipelines → New Pipeline
- Where is your code? → Select GitHub
- Authorize Azure DevOps to access your GitHub account
- Select your repo: cicd-demo → Click 'Approve and install'

- Select 'Starter pipeline' — this creates azure-pipelines.yml in your repo

✓ EXPECTED OUTPUT

```
First pipeline run auto-triggers
Pipeline shows 'Job succeeded' with default hello world steps
```

STEP 6 Write the Full CI/CD Pipeline YAML

Replace the contents of azure-pipelines.yml in your repo with:

```yaml
trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  nodeVersion: '18.x'

stages:
  # ─── STAGE 1: Build & Test ──────────────────
  - stage: Build
    displayName: 'Build and Test'
    jobs:
      - job: BuildJob
        steps:
          - task: NodeTool@0
            inputs:
              versionSpec: $(nodeVersion)
            displayName: 'Install Node.js'

          - script: npm install
            displayName: 'Install dependencies'

          - script: npm test
            displayName: 'Run unit tests'

          - task: ArchiveFiles@2
            inputs:
              rootFolderOrFile: '$(Build.SourcesDirectory)'
              includeRootFolder: false
              archiveFile: '$(Build.ArtifactStagingDirectory)/app.zip'
```

```
            displayName: 'Zip application'

          - task: PublishBuildArtifacts@1
            inputs:
              pathToPublish: '$(Build.ArtifactStagingDirectory)'
              artifactName: 'drop'
            displayName: 'Publish artifact'

  # ─── STAGE 2: Deploy ──────────────────────────
  - stage: Deploy
    displayName: 'Deploy to Azure App Service'
    dependsOn: Build
    condition: succeeded()
    jobs:
      - deployment: DeployJob
        environment: 'production'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
                  inputs:
                    azureSubscription: 'AzureServiceConnection'
                    appType: 'webAppLinux'
                    appName: 'cicd-demo-app-YOUR_INITIALS'
                    package: '$(Pipeline.Workspace)/drop/app.zip'
                  displayName: 'Deploy to Azure App Service'
```

Create the Azure Service Connection (required for deployment):

- Azure DevOps → Project Settings (bottom left) → Service Connections
- New Service Connection → Azure Resource Manager → Service Principal (automatic)
- Select your Azure Subscription → Name it: AzureServiceConnection → Save

Commit and push the pipeline file:

```
git add azure-pipelines.yml
git commit -m "Add CI/CD pipeline with build, test, and deploy stages"
git push origin main
```

✓ EXPECTED OUTPUT

```
Pipeline triggers automatically on push
Stage 1 Build: ✓ Install Node → ✓ npm install → ✓ npm test → ✓ Archive
Stage 2 Deploy: ✓ Deploy to Azure App Service
Open https://cicd-demo-app-YOUR_INITIALS.azurewebsites.net → Hello from
Azure DevOps Pipeline! v1
```

STEP 7 Verify Automatic Deployment on Code Change

Change the response in app.js:

```
app.get('/', (req, res) => res.send('Hello from Azure DevOps Pipeline! v2 - Auto deployed!'));
```

```
git add app.js
git commit -m "Update greeting to v2"
git push origin main
```

✓ EXPECTED OUTPUT

```
Pipeline auto-triggers within seconds of push
Build stage passes tests
Deploy stage updates the app
Visiting Azure URL shows: 'Hello from Azure DevOps Pipeline! v2 - Auto deployed!'
Total time from push to live: ~2-4 minutes
```

PROJECT 2: Docker + Kubernetes on Azure AKS

Containerize a frontend + backend app and orchestrate it on Azure Kubernetes Service.

STEP 1 Create the Python Flask Backend API

Create folder k8s-demo/backend/ and add app.py:

```
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

items = [
  {'id': 1, 'name': 'Azure AKS',    'type': 'Service'},
  {'id': 2, 'name': 'Kubernetes',   'type': 'Orchestrator'},
  {'id': 3, 'name': 'Docker',       'type': 'Container Runtime'},
]

@app.route('/api/items')
def get_items():
    return jsonify(items)

@app.route('/health')
def health():
    return jsonify({'status': 'ok'})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Create backend/requirements.txt:

```
flask==3.0.0
flask-cors==4.0.0
gunicorn==21.2.0
```

Create backend/Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

```
cd k8s-demo/backend
docker build -t backend:v1 .
docker run -d -p 5000:5000 backend:v1
curl http://localhost:5000/api/items
```

✓ EXPECTED OUTPUT

```
[{"id":1,"name":"Azure AKS","type":"Service"}, ...]
3 items returned as JSON
```

STEP 2 Create the Nginx Frontend

Create k8s-demo/frontend/index.html:

```
<!DOCTYPE html><html><head>
<title>K8s Demo App</title>
<style>body{font-family:Arial;max-width:600px;margin:40px auto;padding:20px}
table{width:100%;border-collapse:collapse}
th,td{border:1px solid #ccc;padding:10px;text-align:left}
th{background:#2E75B6;color:white}</style>
</head><body>
<h1>DevOps Items from Backend API</h1>
<table id='items-table'>
  <tr><th>ID</th><th>Name</th><th>Type</th></tr>
</table>
<script>
  const BACKEND = window.BACKEND_URL || 'http://localhost:5000';
  fetch(BACKEND + '/api/items')
    .then(r => r.json())
    .then(items => {
      const t = document.getElementById('items-table');
      items.forEach(i => {
        const row = t.insertRow();
        row.innerHTML = `<td>${i.id}</td><td>${i.name}</td><td>${i.type}</td>`;
      });
    });
</script></body></html>
```

Create frontend/Dockerfile:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

STEP 3 Test Both Containers with Docker Compose

Create k8s-demo/docker-compose.yml at the root:

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - '5000:5000'
    networks:
      - app-network

  frontend:
    build: ./frontend
    ports:
      - '80:80'
    depends_on:
      - backend
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

```
docker-compose up --build
# Open browser: http://localhost → table shows 3 items from backend
```

✓ EXPECTED OUTPUT

```
frontend_1 | /docker-entrypoint.sh: Configuration complete
backend_1  | Booting worker with pid: 7
Browser shows table with 3 DevOps items
```

STEP 4 Push Images to Docker Hub (or Azure ACR)

```
# Login to Docker Hub
docker login

# Tag and push backend
docker tag backend:v1 YOUR_DOCKERHUB_ID/k8s-backend:v1
```

```
docker push YOUR_DOCKERHUB_ID/k8s-backend:v1

# Build frontend and push
cd frontend && docker build -t frontend:v1 .
docker tag frontend:v1 YOUR_DOCKERHUB_ID/k8s-frontend:v1
docker push YOUR_DOCKERHUB_ID/k8s-frontend:v1
```

✓ EXPECTED OUTPUT

```
v1: digest: sha256:abc... size: 123
Images visible at hub.docker.com/r/YOUR_DOCKERHUB_ID/
```

STEP 5 Test on Minikube Locally First

```
minikube start --driver=docker
kubectl get nodes
# Should show: NAME       STATUS   ROLES    AGE
#              minikube   Ready    master   30s
```

STEP 6 Write Kubernetes YAML Files

Create k8s/backend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: YOUR_DOCKERHUB_ID/k8s-backend:v1
          ports:
            - containerPort: 5000
```

```
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 15
          resources:
            requests:
              memory: '64Mi'
              cpu: '50m'
            limits:
              memory: '128Mi'
              cpu: '100m'
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
```

Create k8s/frontend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: YOUR_DOCKERHUB_ID/k8s-frontend:v1
          ports:
            - containerPort: 80
```

```
          resources:
            requests:
              memory: '32Mi'
              cpu: '25m'
            limits:
              memory: '64Mi'
              cpu: '50m'
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
```

Apply all YAML files:

```
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml

# Watch pods come up
kubectl get pods -w

# Check services
kubectl get services

# Local test on Minikube
minikube service frontend-service --url
```

✓ EXPECTED OUTPUT

```
NAME                          READY   STATUS    RESTARTS
backend-5d7c9b8f6-abcde       1/1     Running   0
backend-5d7c9b8f6-fghij       1/1     Running   0
frontend-6b4d9f7c5-klmno      1/1     Running   0
frontend-6b4d9f7c5-pqrst      1/1     Running   0
```

STEP 7 Scale Pods & Perform Rolling Update

Scale backend to 4 replicas and watch:

```
kubectl scale deployment backend --replicas=4
kubectl get pods -w   # Watch 2 new pods appear
kubectl scale deployment backend --replicas=2   # Scale back
```

Perform a rolling update to v2 — build new image, push, then update:

```
# After editing backend app.py (change version to 2.0)
docker build -t YOUR_DOCKERHUB_ID/k8s-backend:v2 ./backend
docker push YOUR_DOCKERHUB_ID/k8s-backend:v2

# Rolling update — zero downtime
kubectl set image deployment/backend backend=YOUR_DOCKERHUB_ID/k8s-backend:v2

# Watch rolling update in real time
kubectl rollout status deployment/backend

# If something goes wrong, rollback instantly
kubectl rollout undo deployment/backend
```

✓ EXPECTED OUTPUT

```
Waiting for deployment 'backend' rollout to finish...
1 out of 2 new replicas have been updated...
2 out of 2 new replicas have been updated...
deployment 'backend' successfully rolled out
```

STEP 8 Deploy to Azure AKS

```
# Create AKS cluster (takes ~5 minutes)
az group create --name k8s-rg --location eastus

az aks create \
  --resource-group k8s-rg \
  --name myAKSCluster \
  --node-count 2 \
  --node-vm-size Standard_B2s \
  --generate-ssh-keys

# Connect kubectl to AKS
az aks get-credentials --resource-group k8s-rg --name myAKSCluster

# Verify connection
kubectl get nodes

# Deploy everything
```

```
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml

# Get external IP (takes ~2 min for Azure to provision)
kubectl get service frontend-service --watch
```

✓ EXPECTED OUTPUT

```
NAME                TYPE           CLUSTER-IP    EXTERNAL-IP      PORT(S)
frontend-service    LoadBalancer   10.0.45.123   20.85.xxx.xxx
80:31234/TCP


Open browser: http://20.85.xxx.xxx → App running on Azure AKS!
```

⚠ NOTE: AKS nodes cost money. Run: az aks delete --name myAKSCluster --resource-group k8s-rg --yes when done.

PROJECT 3: Terraform + Azure Infrastructure as Code

Provision an entire Azure network + VM with Nginx — 100% code, zero portal clicks.

STEP 1 Authenticate Azure & Create Project Structure

```
az login
az account show   # Confirm correct subscription

# Create project folder
mkdir terraform-azure && cd terraform-azure

# Create file structure
touch main.tf variables.tf outputs.tf terraform.tfvars
mkdir modules
```

✓ EXPECTED OUTPUT

```
terraform-azure/
  main.tf  variables.tf  outputs.tf  terraform.tfvars  modules/
```

STEP 2 Configure Provider in main.tf

Add to main.tf:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

```
terraform init
```

✓ EXPECTED OUTPUT

```
Initializing the backend...
Initializing provider plugins...
```

```
- Installing hashicorp/azurerm v3.xx.x...
Terraform has been successfully initialized!
```

STEP 3 Define Variables

Add to variables.tf:

```
variable "resource_group_name" {
  description = "Name of the Azure Resource Group"
  type        = string
  default     = "terraform-demo-rg"
}

variable "location" {
  description = "Azure region"
  type        = string
  default     = "East US"
}

variable "vm_size" {
  description = "Size of the Azure VM"
  type        = string
  default     = "Standard_B1s"
}

variable "admin_username" {
  description = "VM admin username"
  type        = string
  default     = "azureuser"
}

variable "ssh_public_key_path" {
  description = "Path to your SSH public key"
  type        = string
  default     = "~/.ssh/id_rsa.pub"
}
```

Add to terraform.tfvars (override defaults here):

```
resource_group_name = "terraform-demo-rg"
location            = "East US"
vm_size             = "Standard_B1s"
admin_username      = "azureuser"
```

STEP 4 Create Resource Group & Virtual Network

Add to main.tf (below the provider block):

```
# Resource Group
resource "azurerm_resource_group" "main" {
  name     = var.resource_group_name
  location = var.location
  tags = {
    Environment = "dev"
    Project     = "terraform-demo"
    Owner       = "your-name"
  }
}

# Virtual Network
resource "azurerm_virtual_network" "main" {
  name                = "demo-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = azurerm_resource_group.main.tags
}

# Subnet
resource "azurerm_subnet" "main" {
  name                 = "demo-subnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

```
terraform plan    # Preview changes (should show 3 resources to add)
terraform apply   # Type 'yes' to confirm
```

✓ EXPECTED OUTPUT

```
azurerm_resource_group.main: Creating...
azurerm_resource_group.main: Creation complete after 3s
azurerm_virtual_network.main: Creation complete after 5s
azurerm_subnet.main: Creation complete after 4s
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

STEP 5 Create Network Security Group with Rules

Add to main.tf:

```
# Network Security Group
resource "azurerm_network_security_group" "main" {
  name                = "demo-nsg"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  security_rule {
    name                       = "allow-ssh"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name                       = "allow-http"
    priority                   = 110
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "80"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
  tags = azurerm_resource_group.main.tags
}

# Associate NSG with Subnet
resource "azurerm_subnet_network_security_group_association" "main" {
  subnet_id                 = azurerm_subnet.main.id
  network_security_group_id = azurerm_network_security_group.main.id
}
```

STEP 6 Create Public IP, NIC, and Linux VM

Add to main.tf:

```
# Public IP
resource "azurerm_public_ip" "main" {
  name                = "demo-public-ip"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = azurerm_resource_group.main.tags
}

# Network Interface
resource "azurerm_network_interface" "main" {
  name                = "demo-nic"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.main.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.main.id
  }
  tags = azurerm_resource_group.main.tags
}

# Linux Virtual Machine
resource "azurerm_linux_virtual_machine" "main" {
  name                = "demo-vm"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  size                = var.vm_size
  admin_username      = var.admin_username
  disable_password_authentication = true

  network_interface_ids = [azurerm_network_interface.main.id]

  admin_ssh_key {
    username   = var.admin_username
    public_key = file(var.ssh_public_key_path)
  }
```

```
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  # Install Nginx automatically on first boot
  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl start nginx
    systemctl enable nginx
    echo '<h1>Hello from Terraform-provisioned Azure VM!</h1>' >
/var/www/html/index.html
    EOF
  )

  tags = azurerm_resource_group.main.tags
}
```

```
# If no SSH key exists, generate one first:
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa

terraform plan   # Preview all resources
terraform apply  # Creates everything (~3-5 min)
```

✓ EXPECTED OUTPUT

```
azurerm_public_ip.main: Creation complete
azurerm_network_interface.main: Creation complete
azurerm_linux_virtual_machine.main: Creation complete after 45s
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

STEP 7 Add Outputs & Verify Nginx

Add to outputs.tf:

```
output "public_ip_address" {
```

```
  description = "Public IP of the VM"
  value       = azurerm_public_ip.main.ip_address
}

output "ssh_command" {
  description = "SSH command to connect to the VM"
  value       = "ssh ${var.admin_username}@${azurerm_public_ip.main.ip_address}"
}

output "nginx_url" {
  description = "URL to access Nginx"
  value       = "http://${azurerm_public_ip.main.ip_address}"
}
```

```
terraform apply   # Re-apply to show outputs
terraform output  # View all outputs anytime

# SSH into your VM
ssh azureuser@$(terraform output -raw public_ip_address)

# Test Nginx from your machine
curl http://$(terraform output -raw public_ip_address)
```

✓ EXPECTED OUTPUT

```
Outputs:
  public_ip_address = "20.96.xxx.xxx"
  ssh_command       = "ssh azureuser@20.96.xxx.xxx"
  nginx_url         = "http://20.96.xxx.xxx"

curl → <h1>Hello from Terraform-provisioned Azure VM!</h1>
```

STEP 8 Configure Remote State in Azure Blob Storage

Create storage for Terraform state (run once, manually):

```
# Create storage account for remote state
az group create --name terraform-state-rg --location eastus

az storage account create \
  --name tfstate$RANDOM \
  --resource-group terraform-state-rg \
  --sku Standard_LRS \
  --encryption-services blob
```

```
# Note the storage account name printed above (e.g., tfstate12345)

az storage container create \
  --name tfstate \
  --account-name tfstate12345
```

Add remote backend to main.tf (inside the terraform block):

```
terraform {
  required_providers { ... }

  backend "azurerm" {
    resource_group_name  = "terraform-state-rg"
    storage_account_name = "tfstate12345"   # your actual name
    container_name       = "tfstate"
    key                  = "terraform-demo.tfstate"
  }
}
```

```
terraform init -reconfigure   # Migrates local state to Azure Blob
```

✓ EXPECTED OUTPUT

```
Initializing the backend...
Successfully configured the backend 'azurerm'!
Terraform will automatically use this backend unless the backend config changes.
State now stored in Azure Blob — safe for team use!
```

STEP 9 Destroy Everything (Avoid Azure Charges)

```
terraform destroy
# Type 'yes' when prompted

# Verify in Azure Portal or via CLI
az resource list --resource-group terraform-demo-rg --output table

# Delete the resource group completely
az group delete --name terraform-demo-rg --yes --no-wait
```

✓ EXPECTED OUTPUT

```
Destroy complete! Resources: 7 destroyed.
Azure Portal: Resource group empty / deleted
No more charges on your account
```

> ⚠ NOTE: Always destroy after practice. The Standard_B1s VM costs ~$7/month if left running.

Final Checklist — After All 3 Projects

Before adding to your resume or LinkedIn, make sure each project has:

GitHub Repository Standards

- Clean README.md with: What it does, Architecture diagram (even hand-drawn), Tools/Tech used, How to run it locally, Screenshots of working output
- No hardcoded secrets — use environment variables or pipeline variable groups
- Meaningful commit history — not just 'initial commit'
- .gitignore properly set (no node_modules, .terraform, .env)

Practice Explaining Each Project (Interview Prep)

Practice this 2-minute answer structure for each project:

```
"I built a [project name] that [what it does].
 I used [tools] to [specific action].
 The challenge was [problem you solved].
 I learned [key concept] from this project."
```

Additional Projects to Level Up After These 3

- Add monitoring to Project 1: Connect Azure App Service to Azure Monitor + set alerts on error rate
- Add Ingress Controller to Project 2: Deploy nginx-ingress on AKS to route /api to backend and / to frontend
- Add Terraform modules to Project 3: Refactor networking and compute into separate reusable modules
- Link all 3 projects together: Use Terraform to provision AKS cluster, then deploy the Docker app using Azure DevOps pipeline
- Add Azure Key Vault: Store secrets (DB passwords, API keys) in Key Vault and reference from both pipelines and Terraform

Push everything to GitHub. Build it. Break it. Fix it. That's real DevOps.